



TED UNIVERSITY

CMPE 492 – Senior Project II Low Level Design Report

Team Members

Tolga AKYOL

Yiğit AYDIN

Yusuf KAYNAKCI

Supervisor

Dr. Evren Coşkun

Jury Members

Prof. Dr. Tansel Dökeroğlu, Dr. Deniz Cantürk

Document Version: v1.0, Date: 20/03/2026

SmartLeaf

1. Introduction

SmartLeaf is an AI-based plant health analysis system designed to assist users in evaluating plant conditions through leaf images. The system allows users to upload an image of a plant leaf and receive automated outputs including disease classification, confidence scores, health scores, and potential nutrient deficiency indications.

During the CMPE 491 course, a high-level design of the SmartLeaf system was developed, including system architecture, subsystem decomposition, and overall control flow. In CMPE 492, this design is refined into a detailed low-level design.

The purpose of this document is to describe the internal structure of the system in detail, including class-level design, interfaces, data flow, and implementation-related decisions. This document bridges the gap between abstract system design and actual implementation by specifying how each subsystem will be realized.

The report also explains design trade-offs, engineering standards, and the use of design patterns that guide the development of SmartLeaf.

2. Object Design Trade-offs

During the low-level design phase, several design decisions and trade-offs were made in order to balance system performance, scalability, maintainability, and implementation complexity. These decisions are described below.

2.1 Separation of Frontend and Backend

The system follows a clear separation between frontend and backend components. This design improves modularity and maintainability, allowing independent development and testing of user interface and business logic. While integrating both layers into a single monolithic application could reduce communication overhead, it would significantly decrease flexibility and scalability. Therefore, a decoupled architecture was preferred.

2.2 RESTful API vs Alternative Communication Protocols

The backend exposes functionality through RESTful APIs. REST was chosen due to its simplicity, statelessness, and wide compatibility with web technologies. Alternative approaches such as GraphQL or gRPC were considered; however, REST was selected because it is easier to implement, debug, and integrate within the scope of this project.

2.3 Machine Learning Subsystem Isolation

The machine learning component is designed as a separate subsystem rather than being directly embedded into the backend logic. This separation allows independent development, testing, and replacement of the ML model without affecting other parts of the system. While embedding the model directly into the backend could reduce latency, it would reduce flexibility and increase system complexity.

2.4 Model Selection Trade-off

Convolutional Neural Networks (CNNs) were selected for image classification due to their strong performance in computer vision tasks. Lightweight architectures such as MobileNet and EfficientNet were considered to balance accuracy and computational cost. More complex models could provide higher accuracy but would increase training and inference time, making them less suitable for real-time web-based usage.

2.5 Data Storage Strategy

The system minimizes persistent storage of uploaded images. Images are processed temporarily and deleted after analysis unless explicitly saved. This approach reduces storage costs and privacy risks. While permanent storage could enable historical analysis, it introduces additional complexity and data protection concerns.

2.6 Modularity vs Performance

The system is designed with a modular architecture consisting of frontend, backend, and machine learning layers. This improves extensibility and maintainability. However, modular systems introduce additional communication overhead between components. Despite this, modularity was prioritized to support future scalability and independent development of system components.

Overall, the design decisions aim to achieve a balance between usability, performance, scalability, and simplicity, while remaining feasible within the constraints of a student project.

3. Interface Documentation Guidelines

This section defines the principles and conventions used for documenting interfaces between system components. Clear and consistent interface definitions are essential for ensuring correct communication between subsystems and supporting maintainability.

3.1 General Principles

All interfaces in the SmartLeaf system are designed to be:

- Clear and unambiguous
- Consistent across different components
- Independent of implementation details
- Easy to test and validate

Interfaces are documented by explicitly specifying inputs, outputs, and expected behavior.

3.2 Backend API Interface Conventions

The communication between frontend and backend is handled through RESTful API endpoints over HTTP.

Each API endpoint follows these conventions:

- HTTP methods are used according to their purpose (POST for upload, GET for retrieval)
- Requests and responses are formatted in JSON
- All endpoints return structured and consistent responses
- Error cases are explicitly handled and returned with appropriate status codes

Example API structure:

Endpoint: POST /analyze

Request:

```
{  
  "image": <file>  
}
```

Response:

```
{  
  "disease": "early blight",  
  "confidence": 0.92,  
  "health_score": 78,  
  "nutrient_deficiency": "nitrogen"  
}
```

Error Response:

```
{  
  "error": "Invalid image format"  
}
```

3.3 Internal Backend–ML Interface

The backend communicates with the machine learning subsystem through a well-defined interface.

Input:

- Preprocessed image data

Output:

- Predicted class label
- Confidence score
- Additional analysis results

The ML subsystem is treated as a black-box service with clearly defined input and output structures. This ensures that the model can be updated or replaced without affecting the backend logic.

3.4 Frontend–Backend Interaction

The frontend sends user input (image upload) to the backend and receives structured analysis results. The frontend is responsible only for presentation and user interaction, while all processing logic is handled on the backend.

3.5 Interface Consistency and Validation

All interfaces include validation mechanisms to ensure:

- Correct data formats
- Valid input values
- Proper error handling

This reduces runtime errors and improves system reliability.

4. Engineering Standards

The SmartLeaf system is designed and documented in accordance with established software engineering standards and best practices. These standards ensure consistency, clarity, maintainability, and quality throughout the development process.

4.1 UML (Unified Modeling Language)

UML is used as the primary modeling language to represent the structure and behavior of the system. The following UML diagrams are utilized:

- Class Diagrams to represent system classes, attributes, and relationships
- Sequence Diagrams to illustrate interactions between system components
- Component Diagrams to show subsystem organization and dependencies

These diagrams help visualize the system design and support communication among team members.

4.2 IEEE Documentation Standards

The report structure and documentation style follow IEEE guidelines for software design documentation. This includes:

- Clear section organization
- Formal and consistent writing style
- Use of defined terminology and structured descriptions
- Inclusion of references and glossary

These practices ensure that the report is readable, professional, and aligned with academic expectations.

4.3 RESTful API Standards

The backend API is designed according to REST principles:

- Stateless communication between client and server
- Resource-based endpoints
- Use of standard HTTP methods (GET, POST)
- Use of HTTP status codes for error handling

This standard improves interoperability and simplifies integration with different clients.

4.4 Coding Standards

To ensure code quality and maintainability, the following coding standards are followed:

- Python code follows PEP 8 guidelines
- JavaScript code follows ESLint conventions
- Meaningful naming conventions are used for variables, classes, and methods
- Modular structure is maintained for better readability and extensibility

4.5 Version Control and Collaboration

Git is used as the version control system to manage code changes and collaboration. The project repository is hosted on GitHub, allowing:

- Version tracking
- Branch-based development
- Collaboration between team members

Overall, these engineering standards provide a structured foundation for developing a reliable and maintainable system.

5. Definitions, Acronyms, and Abbreviations

This section provides definitions of key terms, acronyms, and abbreviations used throughout this document to ensure clarity and consistency.

AI (Artificial Intelligence):

The field of computer science focused on creating systems capable of performing tasks that typically require human intelligence.

ML (Machine Learning):

A subset of AI that enables systems to learn patterns from data and make predictions.

CNN (Convolutional Neural Network):

A type of deep learning model commonly used for image classification tasks.

API (Application Programming Interface):

A set of rules and protocols that allows communication between different software components.

Frontend:

The client-side part of the system responsible for user interaction and presentation.

Backend:

The server-side part of the system responsible for application logic and data processing.

REST (Representational State Transfer):

An architectural style for designing networked applications using stateless communication.

Health Score:

A numerical value (0–100) representing the estimated condition of a plant leaf.

Nutrient Deficiency Indicator:

A system output suggesting possible nutrient shortages based on visual features of the leaf.

Inference:

The process of applying a trained machine learning model to new input data to generate predictions.

Subsystem:

A modular part of the system responsible for a specific set of functionalities.

6. Packages

The SmartLeaf system is organized into modular packages to ensure separation of concerns, maintainability, and scalability. Each package encapsulates a specific responsibility within the system.

6.1 Frontend Package

The frontend package is responsible for user interaction and presentation. It is implemented as a web-based interface.

Main responsibilities:

- Handling user input (image upload)
- Sending requests to the backend API
- Displaying analysis results
- Managing navigation and UI components

Example components:

- UploadComponent
- ResultComponent
- NavigationComponent

6.2 Backend Package

The backend package manages application logic, request handling, and communication between the frontend and machine learning subsystem.

Main responsibilities:

- Receiving and validating requests
- Handling image uploads
- Orchestrating communication with the ML subsystem
- Returning structured responses to the frontend

Example modules:

- Controller Layer (handles API endpoints)
- Service Layer (handles business logic)
- Utility Layer (common helper functions)

6.3 Machine Learning Package

The machine learning package is responsible for all AI-related operations.

Main responsibilities:

- Image preprocessing
- Model loading and management
- Disease classification and prediction
- Generating confidence scores and additional outputs

Example modules:

- Preprocessor
- ModelLoader
- Predictor

6.4 Data Management Package

This package handles temporary and optional persistent data operations.

Main responsibilities:

- Managing temporary storage of uploaded images
- Handling optional storage of analysis results
- Managing configuration data

6.5 Reporting Package

The reporting package is responsible for formatting and presenting output data.

Main responsibilities:

- Structuring analysis results
- Generating PDF reports
- Formatting outputs for frontend display

6.6 Package Interaction

The interaction between packages follows a layered architecture:

- The frontend communicates only with the backend
- The backend coordinates all operations
- The machine learning package is accessed only through the backend
- Supporting packages (data management, reporting) are used internally by the backend

This modular organization ensures flexibility, easier debugging, and support for future extensions.

7. Class Interfaces

This section describes the main classes, their attributes, methods, and responsibilities within the SmartLeaf system. The system follows a layered architecture consisting of controller, service, and machine learning layers.

7.1 Controller Layer

Class: ImageController

Responsibilities:

Handles incoming HTTP requests from the frontend and delegates processing to the service layer.

Methods:

- `uploadImage(image: File): Response`
Receives the uploaded image and forwards it to the AnalysisService.
- `getHealthStatus(): Response`
Returns system status information (health-check endpoint).

7.2 Service Layer

Class: AnalysisService

Responsibilities:

Coordinates the analysis workflow between the controller and machine learning components.

Attributes:

- `mlService: MLService`
- `reportService: ReportService`

Methods:

- `analyzeImage(image: File): AnalysisResult`
Orchestrates preprocessing, prediction, and result formatting.
- `validateImage(image: File): Boolean`
Validates file format and size.

Class: ReportService

Responsibilities:

Formats analysis results and prepares output for frontend or PDF export.

Methods:

- formatResult(rawResult: MLResult): AnalysisResult
Converts raw ML output into user-friendly format.
- generatePDF(result: AnalysisResult): PDFFile
Generates a downloadable report.

7.3 Machine Learning Layer

Class: MLService

Responsibilities:

Acts as an interface between the backend and the machine learning model.

Attributes:

- modelLoader: ModelLoader
- preprocessor: Preprocessor

Methods:

- predict(image: File): MLResult
Performs preprocessing and model inference.

Class: ModelLoader (Singleton)

Responsibilities:

Loads and manages the machine learning model instance.

Attributes:

- model: CNNModel

Methods:

- getInstance(): ModelLoader
Returns the single instance of the model loader.
- loadModel(): CNNModel
Loads the trained model into memory.

Class: Preprocessor

Responsibilities:

Prepares image data for model inference.

Methods:

- `resize(image: File): Image`
- `normalize(image: Image): Image`
- `segment(image: Image): Image`

7.4 Data Structures

Class: AnalysisResult

Attributes:

- `disease: String`
- `confidence: Float`
- `healthScore: Integer`
- `nutrientDeficiency: String`

Class: MLResult

Attributes:

- `predictedLabel: String`
- `confidence: Float`

8. Components

This section describes the major components of the SmartLeaf system and their interactions. Each component represents a functional unit that contributes to the overall system behavior.

8.1 Frontend Component

The frontend component is responsible for user interaction and presentation. It provides a web-based interface where users can upload plant leaf images and view analysis results.

Responsibilities:

- Accepting user input (image upload)
- Sending requests to the backend API
- Displaying analysis results in a user-friendly format
- Handling navigation and UI rendering

The frontend communicates exclusively with the backend via RESTful API calls.

8.2 Backend Component

The backend component acts as the central coordinator of the system. It processes incoming requests, validates inputs, and manages communication between subsystems.

Responsibilities:

- Handling API requests
- Validating uploaded images
- Orchestrating analysis workflow
- Communicating with the machine learning component
- Returning structured responses

The backend ensures that all system operations are executed correctly and efficiently.

8.3 Machine Learning Component

The machine learning component is responsible for analyzing plant leaf images and generating predictions.

Responsibilities:

- Preprocessing image data
- Performing model inference
- Generating prediction outputs (disease, confidence)
- Supporting extensibility for future model updates

This component operates as a modular and replaceable subsystem.

8.4 Data Management Component

The data management component handles temporary data storage and optional persistence.

Responsibilities:

- Managing uploaded images during processing
- Handling temporary file storage
- Supporting optional storage of analysis results

The system minimizes long-term data storage to reduce complexity and privacy risks.

8.5 Reporting Component

The reporting component formats analysis results and prepares them for presentation or export.

Responsibilities:

- Structuring output data
- Generating reports (e.g., PDF)
- Ensuring consistent output formatting

8.6 Component Interaction Flow

The overall system flow is as follows:

1. The user uploads an image via the frontend
2. The frontend sends a request to the backend
3. The backend validates the input and forwards it to the ML component
4. The ML component processes the image and returns predictions
5. The backend formats the result using the reporting component
6. The final response is sent back to the frontend

This structured interaction ensures a clear separation of concerns and efficient data flow across the system.

9. Glossary

SmartLeaf:

An AI-based system that analyzes plant leaf images to assess plant health.

Disease Classification:

The process of identifying plant diseases based on image analysis.

Health Score:

A numerical value representing the condition of a plant leaf.

Confidence Score:

A measure indicating how certain the model is about its prediction.

Image Preprocessing:

The process of preparing raw images for machine learning analysis.

Inference:

The process of generating predictions using a trained model.

Backend:

The part of the system responsible for processing data and executing logic.

Frontend:

The part of the system responsible for user interaction and display.

API:

A communication interface between different software components.

10. References

- [1] ACM Code of Ethics and Professional Conduct. Available: <https://www.acm.org/code-of-ethics>
- [2] B. Bruegge and A. H. Dutoit, Object-Oriented Software Engineering: Using UML, Patterns, and Java, 2nd Edition, Prentice Hall, 2004.
- [3] Fielding, R. T., "Architectural Styles and the Design of Network-based Software Architectures," Doctoral dissertation, University of California, Irvine, 2000. (REST)
- [4] Chollet, F., Deep Learning with Python, Manning Publications, 2017.